SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Grama Jyothi Billing Program

Field	Entry
Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Design, Implement and Validate a Reliable Billing Program
GitHub Repository URL	https://github.com/bavadarini28 /CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

TASK 7 – Control Type Conversion and Precision

Aim

To identify conversion points and demonstrate surprising results.

Conversion points in the program: unsigned short → int in current − previous (integer promotion); int → unsigned short when returning (modulo 256 effect); unsigned int × int in the slab arithmetic (the usual arithmetic conversions); (long) cast of the units before multiplying by the rate; long → printf %ld.

Expression	Result	Explanation
150 / 100 * 100	100	Integer division truncates 1.5 to 1 before multiplying.
(int)5000000000L	705032704	Cast to a narrower type keeps only the low 32 bits.
-1 < 5u	0 (false)	−1 is converted to unsigned (4294967295), which is larger than 5.
(int)(2.675*100)	267	2.675 is stored as 2.67499999…, so truncation gives 267, not 268.
(unsigned short)65535 + 1	65536	Promoted to int first, so no wrap-around happens; the wrap occurs only when stored back into 16 bits.

Rounding rule: all amounts are integer paise; if a fractional paise ever arises (e.g. 20% peak surcharge), compute in a finer unit and round half-up once: (x + 500) / 1000. Never round at intermediate steps.

Alternative Concept – everything in float, round once

Accumulate bill = 50.0f + … in float and round at the end. Measured with 275.50 per bill: n = 4,000 → error 0.00; n = 60,000 → float total 16,544,776 vs exact 16,530,000 (error +14,776); n = 800,000 → 218,557,600 vs 220,400,000 (error −1,842,400). Once the total is large, adding 275.5 is below float's resolution (about 16 at 2×10^8), so errors grow roughly linearly with n. Time per addition is O(1) both ways (float 4 bytes, long 8 bytes); integers are slightly cheaper on CPUs without an FPU and are exact.

Result: Conversion points identified; five surprises explained; one rounding rule.